A Linux Bus for SVSM Services: Build New, Reuse VIRTIO, or Both?

Stefano Garzarella <sgarzare@redhat.com>

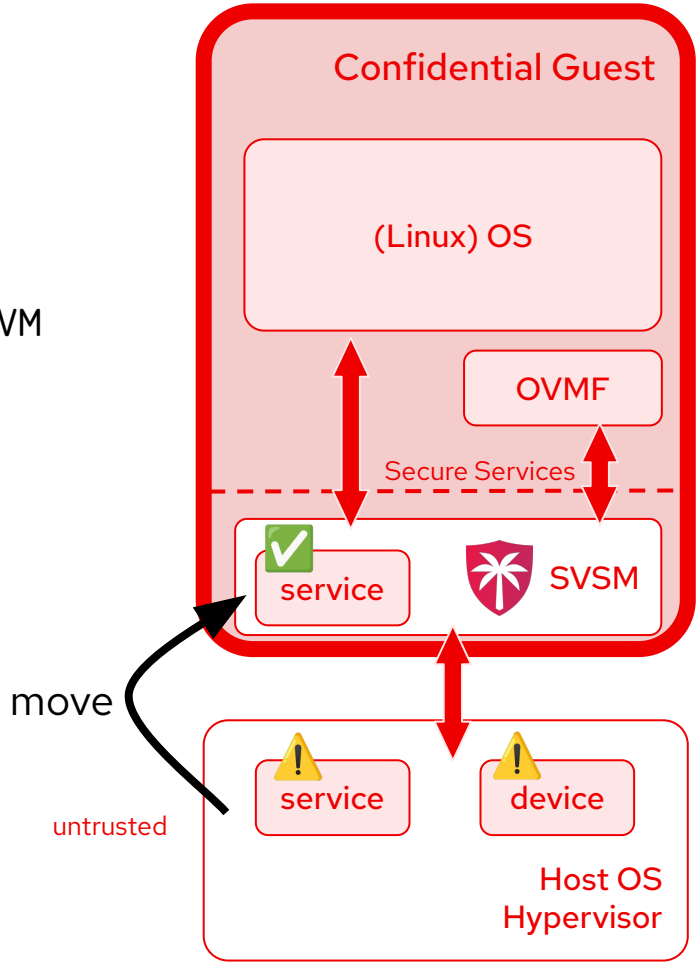
Confidential Computing MC



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

SVSM

- Service Module for confidential VMs:
  - Execution environment for providing services and devices to confidential guest in a secure way
    - Run inside the TCB (Trusted Computing Base) of the CVM
    - E.g. AMD SEV-SNP: SVSM (Secure Vm Service Module)
  - Potential services:
    - Emulating a vTPM
    - UEFI variable storage
    - Migration helper
- https://github.com/coconut-svsm/svsm
  - COCONUT-SVSM



SVSM drivers/services **currently** available in Linux v6.18

1. Trusted Platform Module (TPM) driver

```
// drivers/char/tpm/tpm_svsm.c

/*
 * struct tpm_class_ops {
 *  ...
 *    int (*send)(struct tpm_chip *chip, u8 *buf,
 *                size_t bufsiz, size_t cmd_len);
 *  ...
 * };
 */
static int tpm_svsm_send(struct tpm_chip *chip, u8 *buf,
                         size_t bufsiz, size_t cmd_len)
{
        ...
        ret = snp_svsm_vtpm_send_command(priv->buffer);
        ...
}
```

```
// arch/x86/coco/sev/core.c

/**
 * snp_svsm_vtpm_send_command() - Execute a vTPM operation on SVSM
 * @buffer: A buffer used to both send the command and receive
 *          the response.
 *
 * ...
 */
int snp_svsm_vtpm_send_command(u8 *buffer)
{
    struct svsm_call call = {};

    call.caa = svsm_get_caa();
    call.rax = SVSM_VTPM_CALL(SVSM_VTPM_CMD);
    call.rcx = __pa(buffer);

    return svsm_perform_call_protocol(&call);
}
```

SVSM drivers/services **currently** available in Linux v6.18

2. Trusted Security Module (TSM) Attestation Report Interface

```
// drivers/virt/coco/sev-guest/sev-guest.c

/*
 * struct tsm_report_ops {
 *  ...
 *    int (*report_new)(struct tsm_report *report,
 *                      void *data);
 *  ...
 * };
 */
static int sev_svsm_report_new(struct tsm_report *report,
                               void *data)
{
        ...
        ret = snp_issue_svsm_attest_req(call_id, &call, &ac);
        ...
}
```

```
// arch/x86/coco/sev/core.c

int snp_issue_svsm_attest_req(u64 call_id, struct svsm_call *call,
                              struct svsm_attest_call *input)
{
    ...

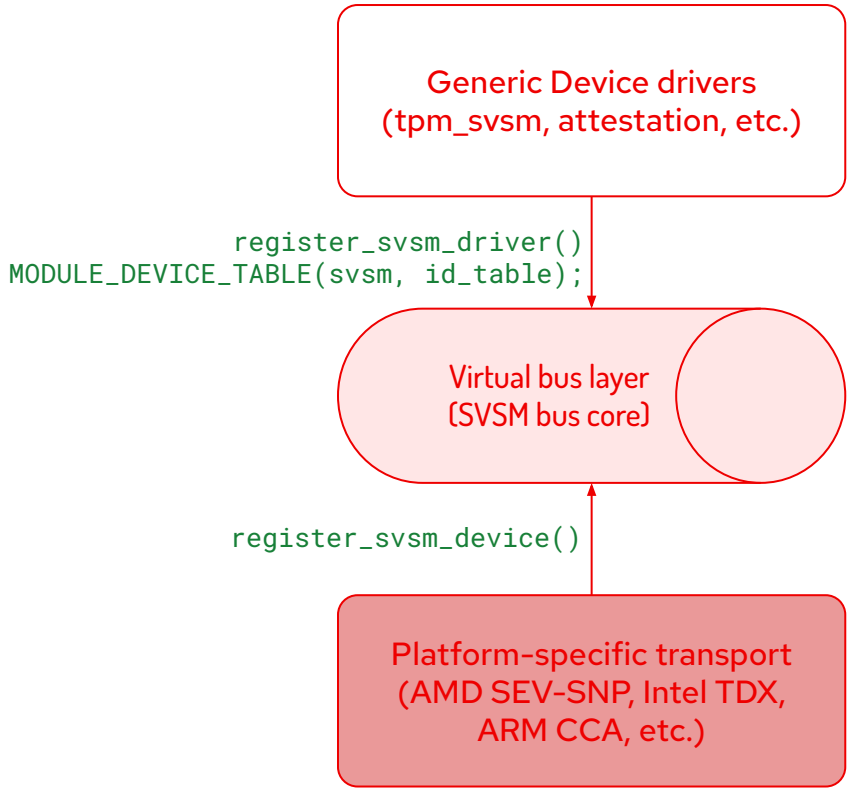
    /*
     * Set input registers for the request and set RDX and R8 to
     * known values in order to detect length values being
     * returned in them.
     */
    call->rax = call_id;
    call->rcx = attest_call_pa;
    call->rdx = -1;
    call->r8 = -1;
    ret = svsm_perform_call_protocol(call);
    update_attest_input(call, input);

    local_irq_restore(flags);

    return ret;
}
```

SVSM Linux bus

- Generic device drivers
  - platform-agnostic
- SVSM bus core
  - common infrastructure to link devices <-> drivers
- Platform-specific transports
  - device discovery
  - provide ops
    - should depends on device type ?
      - `.send_command()`
      - `.read()`
      - `etc.`
    - callbacks assigned during discovery
      - or static C API ?



https://lore.kernel.org/all/38fdf6373fe5f500115d09bcad391e14137b75b8.camel@HansenPartnership.com/

Device discovery

- Platform specific
  - e.g. `snp_init_platform_device()` can discover and register SVSM devices
- Device IDs ?
  - u32
  - strings
  - guid
- Interrupts
  - still open question
  - do we need them?
    - TPM is synchronous

```
// arch/x86/coco/sev/core.c

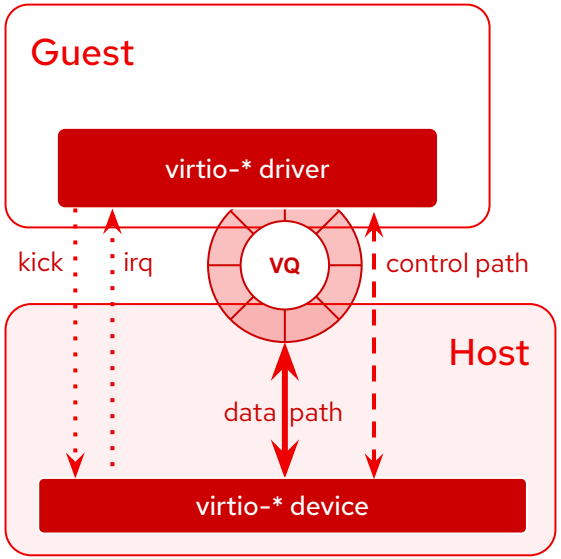
static int __init snp_init_platform_device(void)
{
    if (!cc_platform_has(CC_ATTR_GUEST_SEV_SNP))
        return -ENODEV;

    if (platform_device_register(&sev_guest_device))
        return -ENODEV;

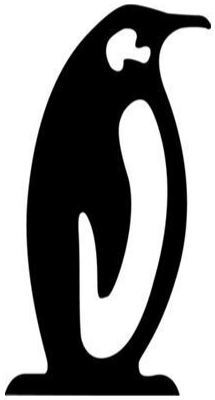
    if (snp_svsm_vtpm_probe() &&
        platform_device_register(&tpm_svsm_device))
        return -ENODEV;

    pr_info("SNP guest platform devices initialized.\n");
    return 0;
}
device_initcall(snp_init_platform_device);
```

VIRTIO



- https://docs.oasis-open.org/virtio/virtio/v1.3/virtio-v1.3.html
- Control path
  - features negotiation
  - configuration space
  - data path setup
- Data path
  - virtqueues
    - split / packed
    - allocated by the guest
- Notifications
  - kick: guest -> host
  - irq: host -> guest
- SVSM
  - [KVM Forum 2025] Would it make sense to support virtqueues to reuse VIRTIO drivers?
    - e.g. adding a new `virtio-svsm` transport
  - Or as potential support for extra devices



東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

